Peak Bloom Prediction Demo

Eager Learner

01/05/2024

Instructions

In this analysis we demonstrate a simple way of predicting the peak bloom date over the next decade for all five locations required by the competition. The models are simple in that they only use the past bloom dates observed at each location—no other covariates or additional information are considered. At the end of this document (Appendix A), we demonstrate a simple way to get historic temperature data for the four locations via the NOAA web API.

For this demo analysis we are using methods from the *tidyverse* of R packages. They can be installed via

```
install.packages('tidyverse')
```

and then loaded via

```
library(tidyverse)
```

Loading the data

The data for the five sites is provided as a simple text file in CSV format. Each file contains the dates of the peak bloom of the cherry trees at the respective site, alongside the geographical location of the site.

The six columns in each data file are

- *location* a human-readable location identifier (`string`).
- *lat* (approximate) latitude of the cherry trees (`double`).
- *long* (approximate) longitude of the cherry trees (`double`).
- *alt* (approximate) altitude of the cherry trees (`double`).
- *year* year of the observation (`integer`).
- *bloom_date* date of peak bloom of the cherry trees (ISO 8601 date `string`). The "peak bloom date" may be defined differently for different sites
- *bloom_doy* days since January 1st of the year until peak bloom (`integer`). January 1st corresponds to `1`.

In R, the data files can be read with `read.csv()` and concatenated with the `bind_rows()` function:

```
cherry <- read.csv("data/washingtondc.csv") |>
  bind_rows(read.csv("data/liestal.csv")) |>
  bind_rows(read.csv("data/kyoto.csv")) |>
  bind_rows(read.csv("data/vancouver.csv")) |>
  bind_rows(read.csv("data/nyc.csv"))
```

For example, the latest 3 observations for each location can be extracted with:

```
cherry |>
  group_by(location) |>
  slice_tail(n = 3)
```

```
## # A tibble: 13 x 7
## # Groups:   location [5]
##    location       lat    long   alt  year bloom_date bloom_doy
##    <chr>        <dbl>   <dbl> <dbl> <int> <chr>          <int>
##  1 kyoto         35.0  136.    44    2022 2022-04-01        91
##  2 kyoto         35.0  136.    44    2023 2023-03-25        84
##  3 kyoto         35.0  136.    44    2024 2024-04-04        95
##  4 liestal       47.5    7.73 350    2023 2023-03-29        88
##  5 liestal       47.5    7.73 350    2024 2024-03-20        80
##  6 liestal       47.5    7.73 350    2025 2025-03-27        86
##  7 newyorkcity   40.7  -74.0    8.5  2024 2024-03-28        88
##  8 vancouver     49.2 -123.    24    2023 2023-04-07        96
##  9 vancouver     49.2 -123.    24    2024 2024-03-23        83
## 10 vancouver     49.2 -123.    24    2024 2024-04-03        93
## 11 washingtondc  38.9  -77.0    0    2022 2022-03-21        80
## 12 washingtondc  38.9  -77.0    0    2023 2023-03-23        82
## 13 washingtondc  38.9  -77.0    0    2024 2024-03-17        77
```

Visualizing the time series

```
cherry |>
  filter(year >= 1880) |>
  ggplot(aes(x = year, y = bloom_doy)) +
  geom_point() +
  geom_step(linetype = 'dotted', color = 'gray50') +
  scale_x_continuous(breaks = seq(1880, 2020, by = 20)) +
  facet_grid(cols = vars(str_to_title(location))) +
  labs(x = "Year", y = "Peak bloom (days since Jan 1st)")
```

Predicting the peak bloom

A simple method to predict peak bloom date in the future is to fit a least-squares line through the observed dates and extrapolate the regression function. We want to have a separate line for each location, hence we tell R to estimate *interaction* effects. We only use data from 1880 to fit the trends, as prior data may not be as reliable/relevant. Since for New York City we only have one observed bloom date, which is not enough to fit a linear regression model, we will omit the site from this simple analysis.

```
# Fit simple least-squares lines for all sites.
cherry_no_nyc <- cherry |>
  filter(location != 'newyorkcity')
ls_fit <- lm(bloom_doy ~ 0 + location + location : year,
             data = cherry_no_nyc, subset = year >= 1880)
```

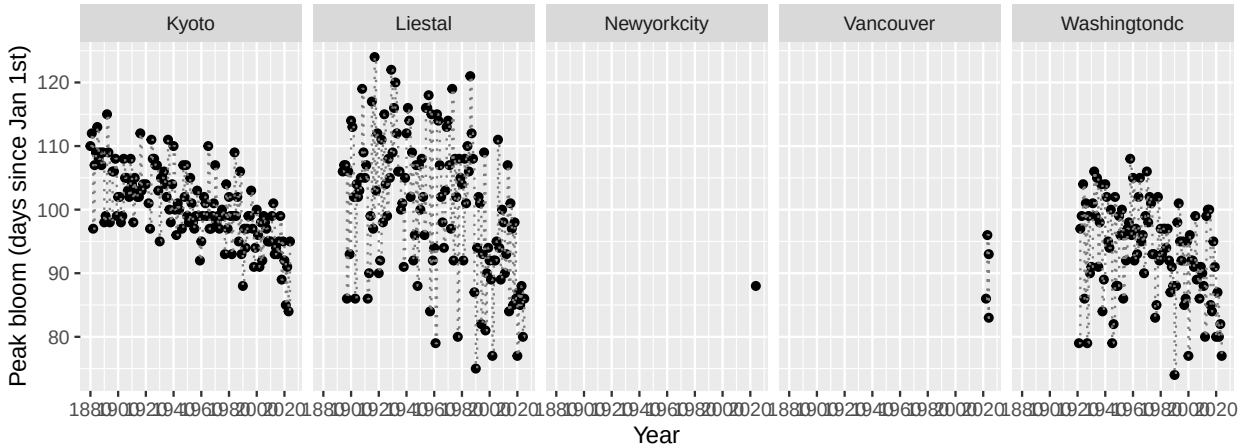


Figure 1: Time series of peak bloom of cherry trees since 1880 at four different sites.

This simple linear regression functions suggest a trend toward earlier peak bloom at the sites. We can compute the actual predictions using the `predict()` function and

```
# Compute the predictions for all 4 sites
cherry_no_nyc_grid <- expand_grid(location = unique(cherry_no_nyc$location),
                                  year = 1880:2025)
predictions <- cherry_no_nyc_grid |>
  bind_cols(predict(ls_fit, newdata = cherry_no_nyc_grid,
                    interval = 'prediction', level = 0.9)) |>
  rename(prediction = fit, lower = lwr, upper = upr)

# Plot the predictions alongside the actual observations for 2015 up to 2023.
cherry |>
  right_join(predictions, by = c('year', 'location')) |>
  filter((location == 'vancouver' & year >= 2021) |
           (location != 'vancouver' & year >= 2015)) |>
  ggplot(aes(x = year, y = prediction, ymin = lower, ymax = upper)) +
  geom_line(linewidth = 1) +
  geom_ribbon(color = 'black', linetype = '22', linewidth = 0.8, fill = NA) +
  geom_point(aes(y = bloom_doy)) +
  scale_x_continuous(breaks = c(2015, 2018, 2021, 2024)) +
  facet_grid(cols = vars(str_to_title(location))) +
  labs(x = "Year", y = "Peak bloom (days since Jan 1st)")
```

Based on this very simple model, the peak bloom dates at these sites are:

```
#' Small helper function to convert the day of year to
#' the actual date.
#'
#' @param year year as an integer
#' @param doy day of the year as integer (1 means January 1st)
#' @return date string
doy_to_date <- function (year, doy) {
  strptime(paste(year, doy, sep = '-'), '%Y-%j') |> # create date object
    strftime('%Y-%m-%d') # translate back to date string in ISO 8601 format
```

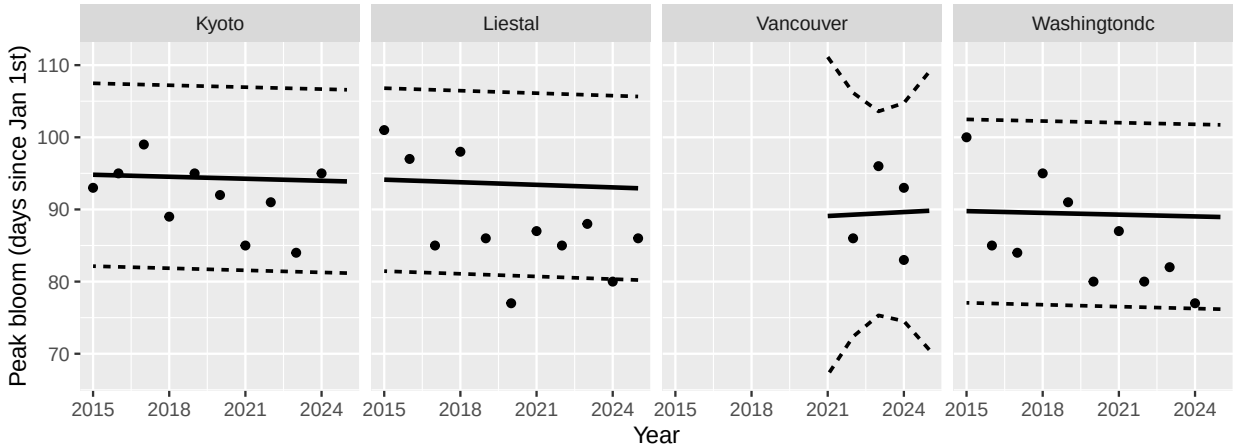


Figure 2: Predictions and 90% prediction intervals from simple linear regression models fitted to four sites.

```
}

predictions |>
  filter(year == 2025) |>
  mutate(prediction = round(prediction),
         lower = floor(lower),
         upper = ceiling(upper),
         prediction_date = doy_to_date(year, prediction))
```

```
## # A tibble: 4 x 6
##   location      year prediction lower upper prediction_date
##   <chr>        <int>      <dbl> <dbl> <dbl> <chr>
## 1 washingtondc  2025         89    76   102 2025-03-30
## 2 liestal       2025         93    80   106 2025-04-03
## 3 kyoto         2025         94    81   107 2025-04-04
## 4 vancouver     2025         90    70   110 2025-03-31
```

Extrapolating to Vancouver, BC and New York City

For the cherry trees in Vancouver, BC and New York City few historical observations are available. This shows in the simple analysis above in the very wide prediction interval. The trees in Vancouver, BC are located approximately at 49.2236916°N (latitude), -123.1636251°E (longitude), 24 meters above sea levels (altitude). Casual observations for Vancouver, BC have been recorded in the way of photos posted to the VCBF Neighbourhood Blog for Kerrisdale. You can search the forum for the keywords "Akebono" (i.e., the name of the cultivar) and "Maple Grove Park" (i.e., the location of the trees).

We need to *extrapolate* from what we have learned about the peak bloom dates in the other locations to Vancouver and NYC. The simple model we have fitted above, however, does not allow us to transfer any knowledge from the other sites – we have only used the history trend at the respective sites.

Although the climate in Vancouver and NYC is different from the other locations, the simplest way to borrow information from the other locations is to average across these three sites. Hence, we want to fit a straight line through the peak bloom dates, ignoring the actual site:

```r
# Fit simple least-squares lines for all sites.
# We use larger weights for observations from Vancouver than for the other sites
ls_fit_for_van <- lm(bloom_doy ~ year, data = cherry, subset = year >= 1880,
                     weights = (location == 'vancouver') +
                       0.2 * (location != 'vancouver'))

vancouver_grid <- tibble(location = 'vancouver', year = 2023:2025)
predictions_vancouver <- vancouver_grid |>
  bind_cols(predict(ls_fit_for_van, newdata = vancouver_grid,
                    interval = 'prediction', level = 0.9)) |>
  rename(prediction = fit, lower = lwr, upper = upr)
```

Not surprisingly, the predicted peak bloom date for Vancouver and NYC is now very similar to the other 3 sites:

```r
predictions_vancouver
```

```
## # A tibble: 3 x 5
##   location   year prediction lower upper
##   <chr>     <int>      <dbl> <dbl> <dbl>
## 1 vancouver  2023       91.2  85.2  97.3
## 2 vancouver  2024       91.1  85.1  97.2
## 3 vancouver  2025       91.0  84.9  97.1
```

We can check the predictions against the data from previous competition years.

```r
# Plot the predictions alongside the actual observations for 2015 up to 2023.
cherry |>
  right_join(predictions_vancouver, by = c('year', 'location')) |>
  ggplot(aes(x = year, y = prediction, ymin = lower, ymax = upper)) +
  geom_line(linewidth = 1) +
  geom_ribbon(color = 'black', linetype = '22', linewidth = 0.8, fill = NA) +
  geom_point(aes(y = bloom_doy)) +
  scale_x_continuous(breaks = 2022:2024) +
  facet_grid(cols = vars(str_to_title(location))) +
  labs(x = "Year", y = "Peak bloom (days since Jan 1st)")
```

If satisfied with the predictions, we can use them instead of the predictions from before.

```r
predictions <- predictions |>
  filter(location != 'vancouver') |>
  bind_rows(predictions_vancouver)
```

Extrapolating to New York City, NY using USA-NPN data

Similar to Vancouver, BC, only few historical observations are available for our location in New York City, NY. There are some historical observations dating back to 2019 in the data provided by USA-NPN. The Washington Square Park has site id 32789 and the Yoshino cherry you should predict has species id 228.

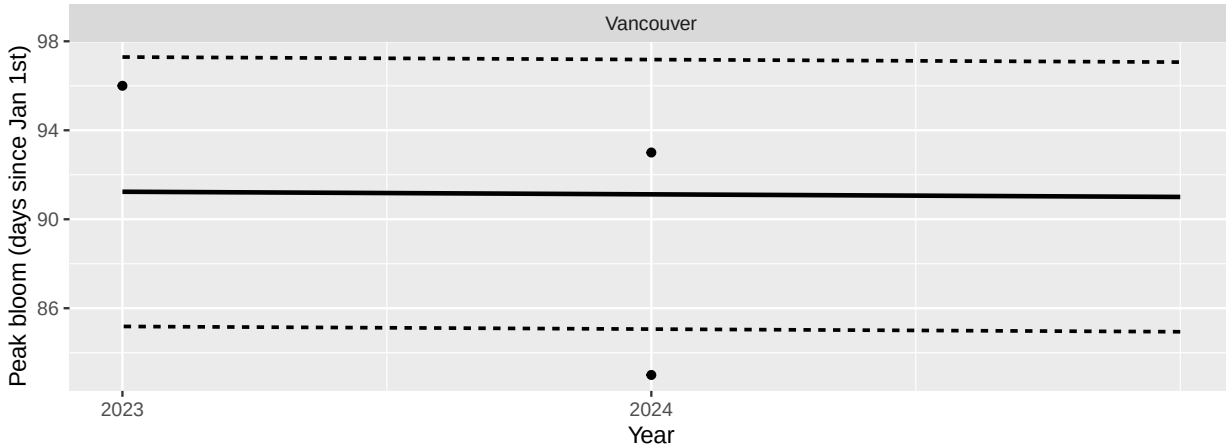


Figure 3: Predictions and 90% prediction intervals from a simple linear regression model for Vancouver using data from all four sites.

```
nyc_data_npn <- read_csv("data/USA-NPN_status_intensity_observations_data.csv") |>
  filter(Site_ID == 32789,
         Species_ID == 228) |>
  mutate(Observation_Date = as_date(Observation_Date, format = '%m/%d/%y'))
```

This data, however, needs to be transformed as it only contains individual observations of the phenophase and not the actual peak bloom date. For simplicity, we take the first day someone observed the flowers to be open as the peak bloom day. This could be done in a more sophisticated way by also looking at the reported intensity value.

```
nyc_data <- nyc_data_npn |>
  arrange(Observation_Date) |>
  mutate(year = year(Observation_Date)) |>
  group_by(year) |>
  summarize(first_flower_index = min(which(Phenophase_Status == 1)),
            bloom_date = strftime(Observation_Date[first_flower_index], format = '%Y-%m-%d'),
            bloom_doy = Day_of_Year[first_flower_index],
            .groups = 'drop') |>
  filter(!is.na(bloom_doy)) |>
  select(-first_flower_index) |>
  mutate(location = 'newyorkcity')

cherry_with_nyc <- cherry |>
  bind_rows(nyc_data)
```

For 2020, no bloom was reported, hence the warning.

Using the same steps as for Vancouver, BC, a simple linear model can be fitted.

```
# Fit simple least-squares lines for all sites.
# We use larger weights for observations from NYC than for the other sites
ls_fit_for_nyc <- lm(bloom_doy ~ year, data = cherry_with_nyc, subset = year >= 1880,
                     weights = (location == 'newyorkcity') +
                       0.2 * (location != 'newyorkcity'))
```

```r
nyc_grid <- tibble(location = 'newyorkcity', year = 2022:2025)
predictions_nyc <- nyc_grid |>
  bind_cols(predict(ls_fit_for_nyc, newdata = nyc_grid,
                    interval = 'prediction', level = 0.9)) |>
  rename(prediction = fit, lower = lwr, upper = upr)

predictions_nyc
```

```
## # A tibble: 4 x 5
##   location     year prediction lower upper
##   <chr>       <int>      <dbl> <dbl> <dbl>
## 1 newyorkcity  2022       91.5  85.5  97.5
## 2 newyorkcity  2023       91.4  85.4  97.4
## 3 newyorkcity  2024       91.3  85.2  97.3
## 4 newyorkcity  2025       91.1  85.1  97.2
```

We can check the predictions against the data from previous competition years.

```r
# Plot the predictions alongside the actual observations for 2015 up to 2023.
cherry_with_nyc |>
  right_join(predictions_nyc, by = c('year', 'location')) |>
  ggplot(aes(x = year, y = prediction, ymin = lower, ymax = upper)) +
  geom_line(linewidth = 1) +
  geom_ribbon(color = 'black', linetype = '22', linewidth = 0.8, fill = NA) +
  geom_point(aes(y = bloom_doy)) +
  scale_x_continuous(breaks = 2022:2025) +
  facet_grid(cols = vars(str_to_title(location))) +
  labs(x = "Year", y = "Peak bloom (days since Jan 1st)")
```

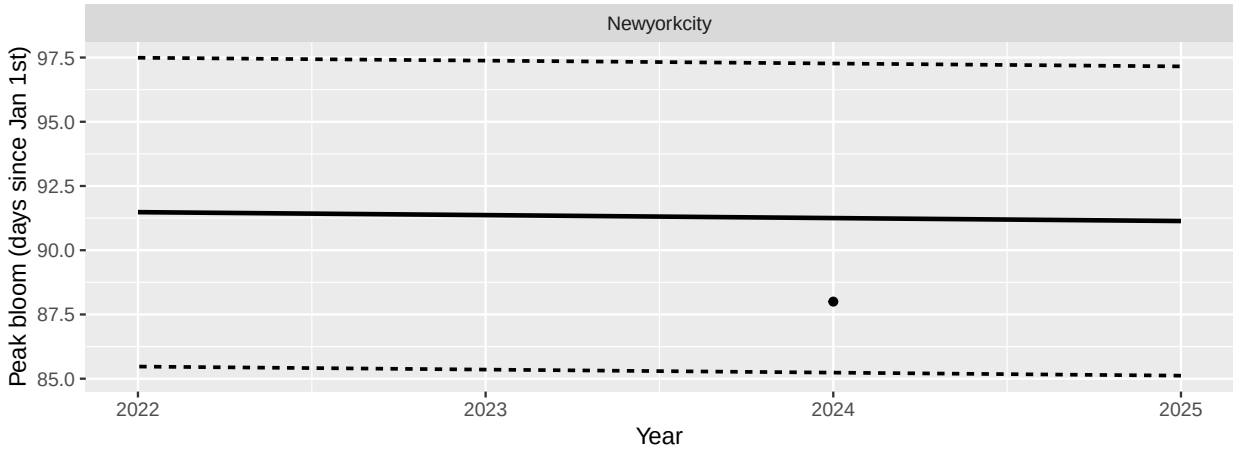


Figure 4: Predictions and 90% prediction intervals from a simple linear regression model for Washington Square Park in NYC using data from all five sites.

If satisfied with the predictions, we can use them instead of the predictions from before.

```r
predictions <- predictions |>
  filter(location != 'newyorkcity') |>
  bind_rows(predictions_nyc)
```

Submission

To submit your entries, enter the predicted bloom dates from your models in the submission form.

```r
predictions |>
  filter(year == 2025) |>
  mutate(predicted_date = strptime(paste(year, prediction), "%Y %j") |>
           as_date())
```

```
## # A tibble: 5 x 6
##   location      year prediction lower upper predicted_date
##   <chr>        <int>      <dbl> <dbl> <dbl> <date>
## 1 washingtondc  2025       89.0  76.2 102.  2025-03-29
## 2 liestal       2025       92.9  80.2 106.  2025-04-02
## 3 kyoto         2025       93.9  81.2 107.  2025-04-03
## 4 vancouver     2025       91.0  84.9  97.1 2025-04-01
## 5 newyorkcity   2025       91.1  85.1  97.2 2025-04-01
```

Appendix: Adding Covariates

We encourage you to find additional publicly-available data that will improve your predictions. For example, one source of global meteorological data comes from the Global Historical Climatology Network (GHCN), available through the NOAA web API.

To use the web API, you first need a web service token. You can request this token (free of charge) via https://www.ncdc.noaa.gov/cdo-web/token. Once you have been issued the token, note it somewhere in your code (or make it available through an environment variable):

```r
NOAA_WEB_API_TOKEN <- 'pHGyMKtJUWWcVVZkAyVDTaXknaQNJqfw'
# or
#NOAA_WEB_API_TOKEN <- Sys.getenv("NOAA_WEB_API_TOKEN")
```

To connect to and use the web API you may use the following R packages:

```r
install.packages("httr2")
install.packages("jsonlite")
```

and the loaded via

```r
library(httr2)
library(jsonlite)
```

The stations closest to the sites for the competition with continuously collected maximum temperatures are USC00186350 (Washington D.C.), GME00127786 (Liestal), JA000047759 (Kyoto), CA001108395 (Vancouver) and

```r
NOAA_API_BASE_URL <- "https://www.ncei.noaa.gov/cdo-web/api/v2/data"

# Define the station IDs for the specified locations
stations <- c(
  "washingtondc" = "GHCND:USW00013743",
  "vancouver"    = "GHCND:CA001108395",
  "newyorkcity"  = "GHCND:USW00014732",
  "liestal"      = "GHCND:SZ000001940",
  "kyoto"        = "GHCND:JA000047759")
```

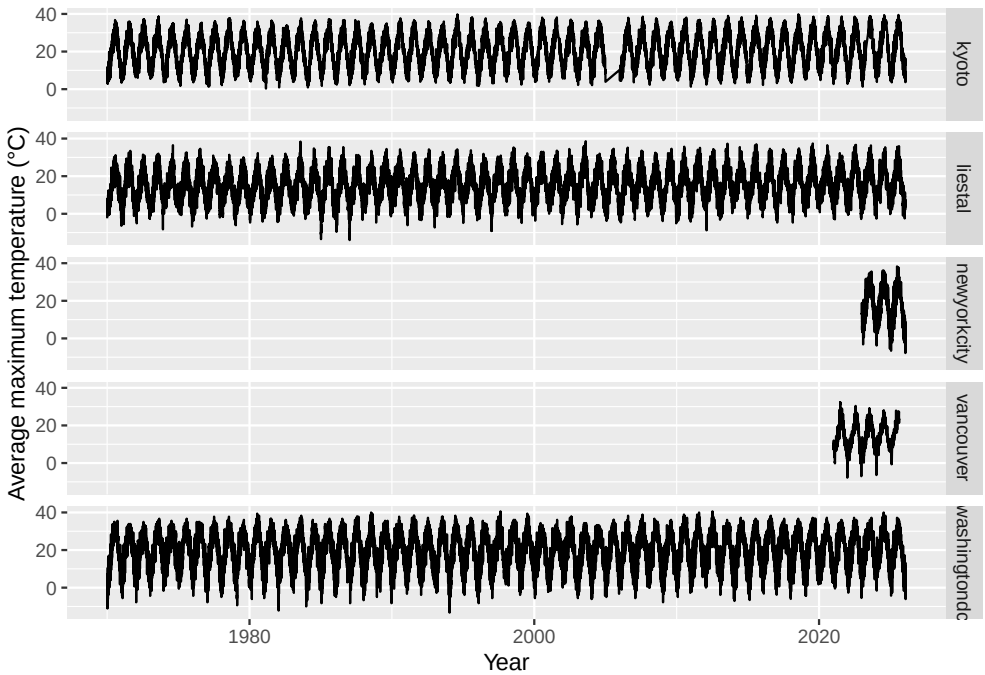
As a simple demonstration, we retrieve the average seasonal maximum daily temperature (in 1/10 °C) from these stations using our own `get_temperature()` function, which wraps the `ghcnd_search()` function in the `rnoaa` package. (N.b. `ghcnd_search()` returns a list. Each element of the list corresponds to an element of the `var` argument.)

```r
nested_to_tibble <- function (x) {
  # Determine the variable names in the response
  variable_names <- map(x, names) |>
    unlist(use.names = FALSE) |>
    unique()

  names(variable_names) <- variable_names

  # Reshape the response from a nested list into a table
  map(variable_names, \(i) {
    map(x, \(y) {
      if (is.null(y[[i]])) {
        NA_character_
      } else {
        y[[i]]
      }
    }) |>
      unlist(use.names = FALSE)
  }) |>
    as_tibble()
}

get_daily_avg_temp <- function(station_id, start_date, end_date,
                               api_key, base_url, window_size = 300) {
  windows <- seq(as_date(start_date),
                 as_date(end_date) + days(window_size + 1),
                 by = sprintf("%d days", window_size))

  batches <- map2(windows[-length(windows)], windows[-1] - days(1), \(from, to) {
    if (from > Sys.Date()) {
      return(NULL)
    }
    tryCatch(
      request(base_url) |>
        req_headers(token = api_key) |>
        req_url_query(
          datasetid = "GHCND",
          stationid = station_id,
```

```r
          datatypeid = "TAVG,TMAX",
          startdate = from,
          enddate = min(as_date(to), Sys.Date()),
          units = "metric",
          limit = 1000
        ) |>
        req_retry(max_tries = 10) |>
        req_perform() |>
        resp_body_json(),

      httr2_http = \(cnd) {
        rlang::warn(sprintf("Failed to retrieve data for station %s in time window %s--%s",
                            station_id, from, to),
                    parent = cnd)
        NULL
      }
    )
  })

  batches <- purrr::compact(batches)

  map(batches, \(x) nested_to_tibble(x$results)) |>
    list_rbind() |>
    mutate(date = as.Date(substr(as.character(unlist(date)), 1, 10)))
}
```

```r
historic_temperatures <- cherry |>
  group_by(location) |>
  summarize(start_date = sprintf('%d-01-01', max(1970, min(year) - 1))) |>
  left_join(tibble(location = names(stations),
                   station_id = stations),
            by = "location") |>
  group_by(location) |>
  group_modify(\(x, gr) {
    get_daily_avg_temp(
      station_id = x$station_id,
      start_date = x$start_date,
      end_date = Sys.Date(),
      api_key = NOAA_WEB_API_TOKEN,
      base_url = NOAA_API_BASE_URL
    )
  })
```

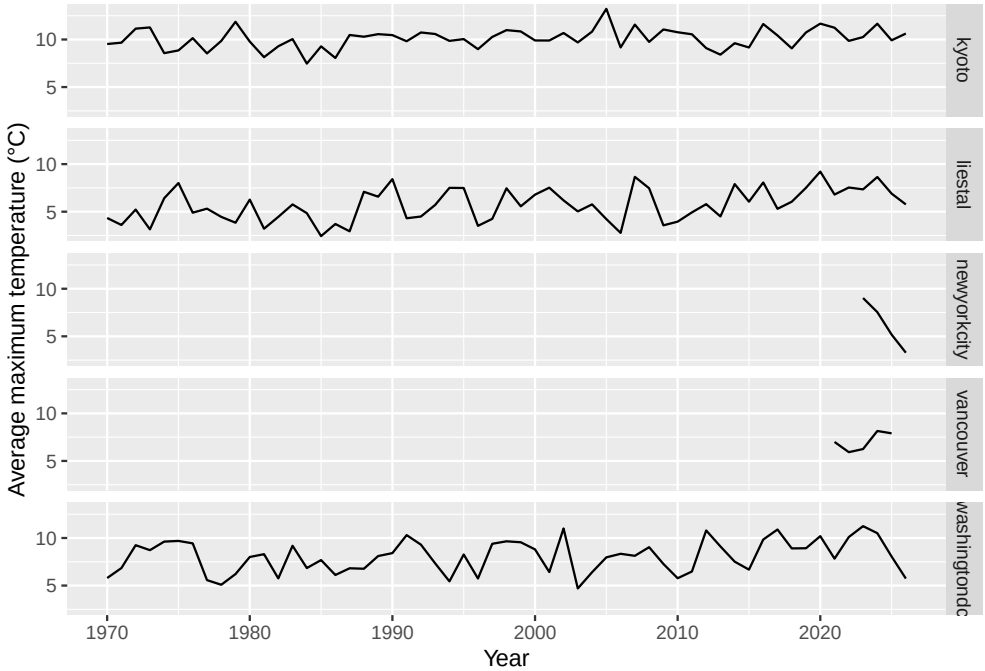
```r
historic_temperatures |>
  filter(datatype == 'TMAX') |>
  ggplot(aes(x = date, y = value)) +
  geom_line() +
  labs(x = "Year", y = "Average maximum temperature (°C)") +
  facet_grid(rows = vars(location))
```



A simple model may simply take the average maximum winter temperature (Dec. 1st of the previous year until end of February) into account:

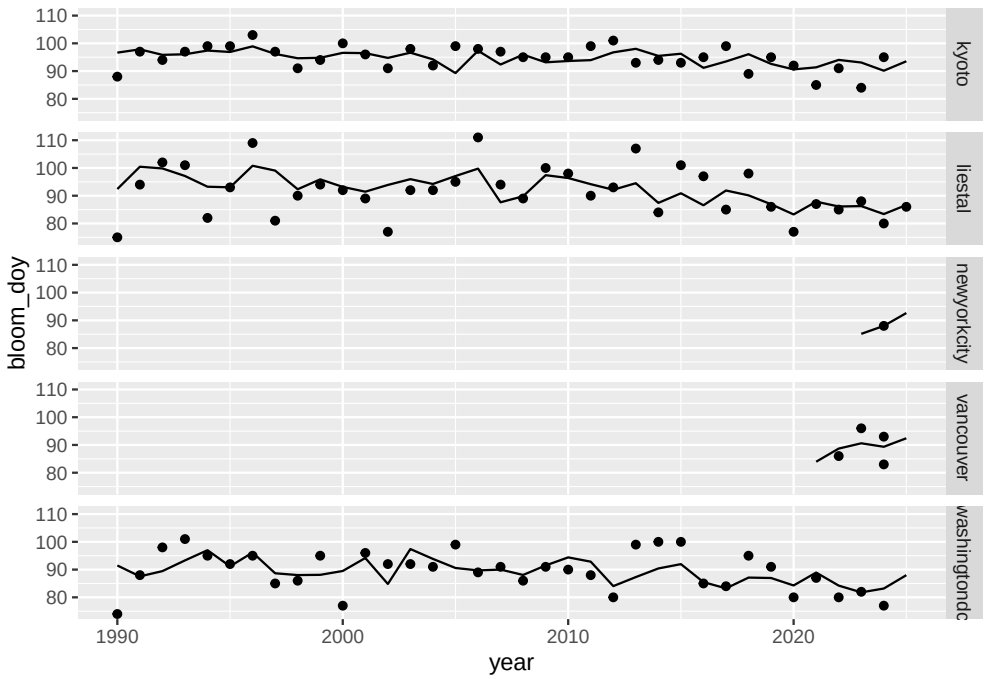
```
avg_winter_temp <- historic_temperatures |>
  filter(datatype == 'TMAX') |>
  mutate(year = case_when(
    month(date) < 3 ~ year(date),
    month(date) == 12 ~ year(date) + 1,
    TRUE ~ NA_integer_
  )) |>
  filter(!is.na(year), year >= 1970) |>
  group_by(location, year) |>
  summarize(avg_tmax = mean(value),
            .groups = 'drop')
```

```
avg_winter_temp |>
  ggplot(aes(x = year, y = avg_tmax)) +
  geom_line() +
  labs(x = "Year", y = "Average maximum temperature (°C)") +
  facet_grid(rows = vars(location))
```



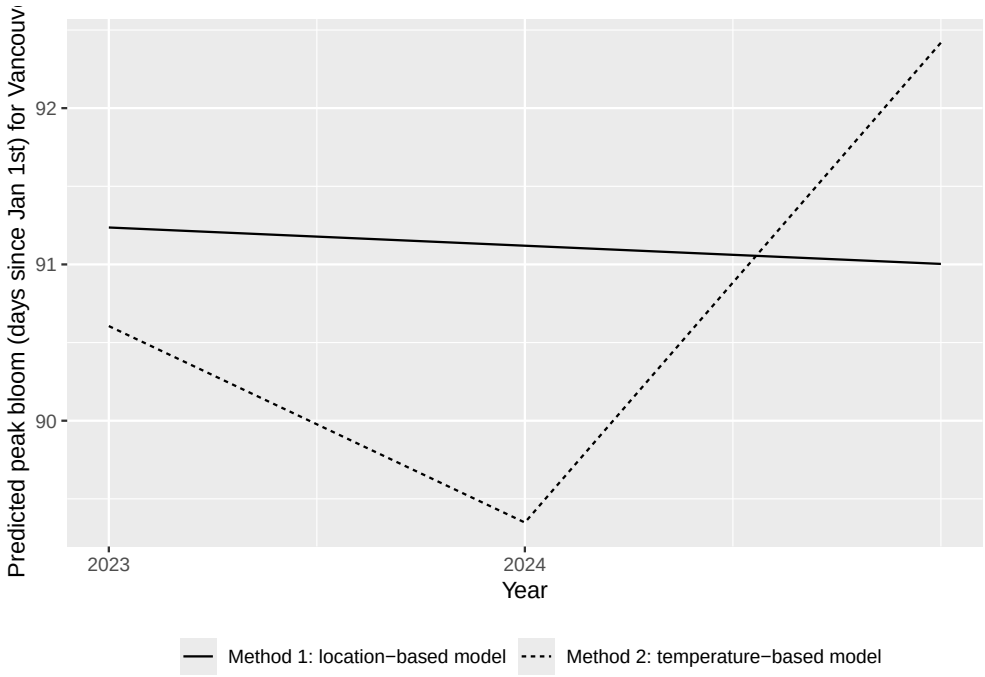
Using these average temperature, we can predict the peak bloom date again using linear regression with location-specific temporal trends and a global effect of average winter temperatures.

```
ls_fit_with_temp <- cherry |>
  inner_join(avg_winter_temp,
             by = c("location", "year")) |>
  lm(formula = bloom_doy ~ year * location + avg_tmax)

cherry_grid <- expand_grid(location = unique(cherry$location),
                           year = 1990:2025) |>
  inner_join(avg_winter_temp,
             by = c("location", "year"))

predictions_from_temp <- cherry_grid |>
  mutate(pred_bloom = predict(ls_fit_with_temp, newdata = cherry_grid))

predictions_from_temp |>
  left_join(cherry,
            by = c("location", "year")) |>
  ggplot(aes(x = year)) +
  geom_point(aes(y = bloom_doy)) +
  geom_line(aes(y = pred_bloom)) +
  facet_grid(rows = vars(location))
```



The following plot shows a comparison of predictions for Vancouver using the two methods described in this demo.

```
predictions_vancouver |>
  left_join(predictions_from_temp,
            by = c("location", "year")) |>
  select(year, pred_temporal = prediction, pred_temp = pred_bloom) |>
  pivot_longer(cols = -year) |>
  mutate(name = if_else(name == "pred_temporal",
                        "Method 1: location-based model",
                        "Method 2: temperature-based model")) |>
  ggplot() +
  aes(x = year, y = value, linetype = name) +
  geom_line() +
  scale_x_continuous(breaks = 2022:2024) +
  labs(x = "Year", linetype = "",
       y = "Predicted peak bloom (days since Jan 1st) for Vancouver") +
  theme(legend.position = "bottom")
```



Linear Regression Model

```
library(tidyverse)

# remove NYC because not enough data
cherry_no_nyc <- cherry |>
  filter(location != "newyorkcity")

# Linear regression with interaction (separate slope for each site)
ls_fit <- lm(bloom_doy ~ 0 + location + location:year,
             data = cherry_no_nyc,
             subset = year >= 1880)

summary(ls_fit)
```

```
##
## Call:
## lm(formula = bloom_doy ~ 0 + location + location:year, data = cherry_no_nyc,
##     subset = year >= 1880)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -22.1603  -4.2156   0.1975   4.4968  23.4243
##
## Coefficients:
##                            Estimate Std. Error t value Pr(>|t|)
## locationkyoto              281.82551   29.77521   9.465  < 2e-16 ***
## locationliestal            333.76763   34.02293   9.810  < 2e-16 ***
## locationvancouver         -278.36364 9272.25562  -0.030   0.9761
```

```
## locationwashingtondc       254.34647   48.97001   5.194 3.39e-07 ***
## locationkyoto:year          -0.09281    0.01524  -6.089 2.83e-09 ***
## locationliestal:year        -0.11893    0.01736  -6.851 3.04e-11 ***
## locationvancouver:year       0.18182    4.58285   0.040   0.9684
## locationwashingtondc:year   -0.08168    0.02482  -3.290   0.0011 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.6 on 373 degrees of freedom
## Multiple R-squared:  0.9942, Adjusted R-squared:  0.9941
## F-statistic:  8023 on 8 and 373 DF,  p-value: < 2.2e-16
```

```r
cherry_no_nyc_grid <- expand_grid(
  location = unique(cherry_no_nyc$location),
  year = 1880:2025
)

predictions <- cherry_no_nyc_grid |>
  bind_cols(predict(ls_fit,
                    newdata = cherry_no_nyc_grid,
                    interval = "prediction",
                    level = 0.9)) |>
  rename(prediction = fit,
         lower = lwr,
         upper = upr)

head(cherry_no_nyc_grid)
```

```
## # A tibble: 6 x 2
##   location      year
##   <chr>        <int>
## 1 washingtondc  1880
## 2 washingtondc  1881
## 3 washingtondc  1882
## 4 washingtondc  1883
## 5 washingtondc  1884
## 6 washingtondc  1885
```

```r
ls_fit_for_van <- lm(
  bloom_doy ~ year,
  data = cherry,
  subset = year >= 1880,
  weights = (location == "vancouver") +
            0.2 * (location != "vancouver")
)

summary(ls_fit_for_van)
```

```
##
## Call:
## lm(formula = bloom_doy ~ year, data = cherry, subset = year >=
##     1880, weights = (location == "vancouver") + 0.2 * (location !=
##     "vancouver"))
```

```
##
## Weighted Residuals:
##      Min       1Q   Median       3Q      Max
## -10.7883  -1.7777   0.1121   2.0102  11.3824
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 326.9957    19.8448   16.48   <2e-16 ***
## year         -0.1165     0.0101  -11.54   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.601 on 380 degrees of freedom
## Multiple R-squared:  0.2594, Adjusted R-squared:  0.2574
## F-statistic: 133.1 on 1 and 380 DF,  p-value: < 2.2e-16
```

```r
vancouver_grid <- tibble(
  location = "vancouver",
  year = 2023:2025
)

predictions_vancouver <- vancouver_grid |>
  bind_cols(predict(ls_fit_for_van,
                    newdata = vancouver_grid,
                    interval = "prediction",
                    level = 0.9)) |>
  rename(prediction = fit,
         lower = lwr,
         upper = upr)

head(vancouver_grid)
```

```
## # A tibble: 3 x 2
##   location   year
##   <chr>     <int>
## 1 vancouver  2023
## 2 vancouver  2024
## 3 vancouver  2025
```

```r
ls_fit_for_nyc <- lm(
  bloom_doy ~ year,
  data = cherry_with_nyc,
  subset = year >= 1880,
  weights = (location == "newyorkcity") +
            0.2 * (location != "newyorkcity")
)

summary(ls_fit_for_nyc)
```

```
##
## Call:
## lm(formula = bloom_doy ~ year, data = cherry_with_nyc, subset = year >=
##     1880, weights = (location == "newyorkcity") + 0.2 * (location !=
```

```
##     "newyorkcity"))
##
## Weighted Residuals:
##      Min       1Q   Median       3Q      Max
## -10.7777  -1.8656   0.1214   2.0063  11.3490
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 324.06947   20.34949   15.93   <2e-16 ***
## year         -0.11503    0.01037  -11.09   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.568 on 380 degrees of freedom
##   (1 observation deleted due to missingness)
## Multiple R-squared:  0.2446, Adjusted R-squared:  0.2426
## F-statistic: 123.1 on 1 and 380 DF,  p-value: < 2.2e-16
```

```r
df <- cherry |> inner_join(avg_winter_temp, by = c("location", "year"))

ls_fit_with_temp <- lm(bloom_doy ~ year * location + avg_tmax, data = df)

summary(ls_fit_with_temp)
```

```
##
## Call:
## lm(formula = bloom_doy ~ year * location + avg_tmax, data = df)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -21.942  -3.408   0.000   3.965  18.084
##
## Coefficients: (1 not defined because of singularities)
##                             Estimate Std. Error t value Pr(>|t|)
## (Intercept)               3.576e+02  1.118e+02   3.198  0.00167 **
## year                     -1.205e-01  5.618e-02  -2.145  0.03345 *
## locationliestal           2.537e+02  1.568e+02   1.618  0.10772
## locationnewyorkcity      -1.044e+01  6.881e+00  -1.517  0.13123
## locationvancouver        -5.451e+03  8.055e+03  -0.677  0.49951
## locationwashingtondc     -9.217e+00  1.579e+02  -0.058  0.95354
## avg_tmax                 -2.015e+00  3.578e-01  -5.631 7.82e-08 ***
## year:locationliestal     -1.317e-01  7.841e-02  -1.679  0.09508 .
## year:locationnewyorkcity         NA         NA      NA       NA
## year:locationvancouver    2.689e+00  3.981e+00   0.676  0.50028
## year:locationwashingtondc -3.817e-05  7.906e-02   0.000  0.99962
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.565 on 161 degrees of freedom
## Multiple R-squared:  0.4211, Adjusted R-squared:  0.3887
## F-statistic: 13.01 on 9 and 161 DF,  p-value: 1.778e-15
```

```r
cherry_grid <- expand_grid(
  location = unique(cherry$location),
  year = 1990:2025
) |>
  inner_join(avg_winter_temp, by = c("location", "year"))

predictions_from_temp <- cherry_grid |>
  mutate(pred_bloom = predict(ls_fit_with_temp,
                              newdata = cherry_grid))

head(cherry_grid)
```

```
## # A tibble: 6 x 3
##   location      year avg_tmax
##   <chr>        <dbl>    <dbl>
## 1 washingtondc  1990     8.41
## 2 washingtondc  1991    10.3
## 3 washingtondc  1992     9.3
## 4 washingtondc  1993     7.33
## 5 washingtondc  1994     5.44
## 6 washingtondc  1995     8.27
```

```r
model <- lm(bloom_doy ~ year * location, data = cherry)
summary(model)
```

```
##
## Call:
## lm(formula = bloom_doy ~ year * location, data = cherry)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -22.1603  -4.6801   0.0931   4.9292  23.4243
##
## Coefficients: (1 not defined because of singularities)
##                              Estimate Std. Error t value Pr(>|t|)
## (Intercept)                1.071e+02  1.275e+00  84.022  < 2e-16 ***
## year                      -1.760e-03  8.051e-04  -2.186 0.029008 *
## locationliestal            2.266e+02  3.194e+01   7.096 2.33e-12 ***
## locationnewyorkcity       -1.557e+01  7.142e+00  -2.181 0.029432 *
## locationvancouver         -3.855e+02  8.697e+03  -0.044 0.964652
## locationwashingtondc       1.472e+02  4.595e+01   3.204 0.001396 **
## year:locationliestal      -1.172e-01  1.630e-02  -7.187 1.24e-12 ***
## year:locationnewyorkcity          NA         NA      NA       NA
## year:locationvancouver     1.836e-01  4.298e+00   0.043 0.965942
## year:locationwashingtondc -7.992e-02  2.330e-02  -3.430 0.000626 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.128 on 1068 degrees of freedom
## Multiple R-squared:  0.2356, Adjusted R-squared:  0.2298
## F-statistic: 41.14 on 8 and 1068 DF,  p-value: < 2.2e-16
```

```r
final_model <- lm(
  bloom_doy ~ year * location,
  data = cherry
)

summary(final_model)
```

```
##
## Call:
## lm(formula = bloom_doy ~ year * location, data = cherry)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -22.1603  -4.6801   0.0931   4.9292  23.4243
##
## Coefficients: (1 not defined because of singularities)
##                             Estimate Std. Error t value Pr(>|t|)
## (Intercept)                1.071e+02  1.275e+00  84.022  < 2e-16 ***
## year                      -1.760e-03  8.051e-04  -2.186 0.029008 *
## locationliestal            2.266e+02  3.194e+01   7.096 2.33e-12 ***
## locationnewyorkcity       -1.557e+01  7.142e+00  -2.181 0.029432 *
## locationvancouver         -3.855e+02  8.697e+03  -0.044 0.964652
## locationwashingtondc       1.472e+02  4.595e+01   3.204 0.001396 **
## year:locationliestal      -1.172e-01  1.630e-02  -7.187 1.24e-12 ***
## year:locationnewyorkcity          NA         NA      NA       NA
## year:locationvancouver     1.836e-01  4.298e+00   0.043 0.965942
## year:locationwashingtondc -7.992e-02  2.330e-02  -3.430 0.000626 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.128 on 1068 degrees of freedom
## Multiple R-squared:  0.2356, Adjusted R-squared:  0.2298
## F-statistic: 41.14 on 8 and 1068 DF,  p-value: < 2.2e-16
```

```r
final_model <- lm(bloom_doy ~ year * location, data = cherry)

summary(final_model)
```

```
##
## Call:
## lm(formula = bloom_doy ~ year * location, data = cherry)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -22.1603  -4.6801   0.0931   4.9292  23.4243
##
## Coefficients: (1 not defined because of singularities)
##                             Estimate Std. Error t value Pr(>|t|)
## (Intercept)                1.071e+02  1.275e+00  84.022  < 2e-16 ***
## year                      -1.760e-03  8.051e-04  -2.186 0.029008 *
## locationliestal            2.266e+02  3.194e+01   7.096 2.33e-12 ***
## locationnewyorkcity       -1.557e+01  7.142e+00  -2.181 0.029432 *
```

```
## locationvancouver         -3.855e+02  8.697e+03  -0.044 0.964652
## locationwashingtondc       1.472e+02  4.595e+01   3.204 0.001396 **
## year:locationliestal      -1.172e-01  1.630e-02  -7.187 1.24e-12 ***
## year:locationnewyorkcity          NA         NA      NA       NA
## year:locationvancouver     1.836e-01  4.298e+00   0.043 0.965942
## year:locationwashingtondc -7.992e-02  2.330e-02  -3.430 0.000626 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.128 on 1068 degrees of freedom
## Multiple R-squared:  0.2356, Adjusted R-squared:  0.2298
## F-statistic: 41.14 on 8 and 1068 DF,  p-value: < 2.2e-16
```

```r
future_2026 <- data.frame(
  location = unique(cherry$location),
  year = 2026
)
```

```r
predictions_2026 <- predict(final_model, newdata = future_2026)

future_2026$predicted_doy <- round(predictions_2026)
```

```r
future_2026$predicted_date <- as.Date(
  paste(future_2026$year, future_2026$predicted_doy),
  format = "%Y %j"
)

future_2026
```

```
##       location year predicted_doy predicted_date
## 1 washingtondc 2026            89     2026-03-30
## 2      liestal 2026            93     2026-04-03
## 3        kyoto 2026           104     2026-04-14
## 4    vancouver 2026            90     2026-03-31
## 5  newyorkcity 2026            88     2026-03-29
```